BACKEND | CONCEPTS & DEMO

Short speaking guide based on the implemented local application

Sparsh Dwiwedi - 261010290 | Backend
Project: NoteSphere - Smart Student Notes Sharing Platform

Your 45-60 second opening

"I am responsible for the backend of NoteSphere. It runs on Node.js using Next.js API route handlers written in TypeScript. The backend validates input with Zod, checks the user session and role, and reads or writes PostgreSQL data. Admins can upload PDFs; students can only access published notes after signing in. Passwords are stored as salted scrypt hashes, and sessions use HttpOnly cookies. PDF files are kept in private local storage, with an authorized API controlling access. The backend also manages reviews, bookmarks, collections and notifications."

The backend stack

TECHNOLOGY	PURPOSE IN THIS APPLICATION
Node.js + Next.js	Node.js is the JavaScript server runtime. Next.js handles HTTP requests through src/app/api/[...path]/route.ts; a separate Express server is not used.
TypeScript + Zod	TypeScript checks code during development. Zod validates request values at runtime: email, password length, note metadata, ratings and review text.
node:crypto	Generates random identifiers/tokens, hashes passwords with salted scrypt and hashes session tokens before database storage.
SQL + database helpers	Parameterized SQL queries use bound values. PGlite runs local PostgreSQL; the pg driver supports an externally configured PostgreSQL server.
file-type + node:fs	The server checks PDF contents and size, then writes private document bytes to disk under random storage keys.

Explain one complete request

Admin publishes a PDF: Upload form sends multipart data to POST /api/upload. The server checks origin, session and admin role, validates metadata and PDF signatures, checks size/hash, saves bytes and inserts note/file records. A published community note generates student notifications. The response updates the UI.

Student reads it: An authenticated request to GET /api/notes/:id/file checks access before returning document bytes. A browser URL by itself does not grant file access.

BACKEND | QUESTIONS & CODE MAP

Use these answers after rehearsing the actual code and demonstration

How do login and logout work?

Login verifies the password against its salted scrypt hash. The server creates a random session token, stores its hash in the sessions table and sets an HttpOnly cookie. Later requests look up the valid session. Logout revokes that session and clears the cookie.

How are admin accounts protected?

Student registration always creates a student. Admin registration requires an unexpired, unused invitation created by an existing admin. The server consumes that invitation atomically; a request cannot assign its own role.

What validation happens for uploads?

Only admins can upload. The server checks the content-detected type, PDF header/EOF, size limit and duplicate content hash. The filename extension and browser MIME are not trusted. New uploads are PDF only; the local limit is 20 MB.

What do 401, 403 and 404 mean here?

401 means sign-in is required. 403 means a role or request-origin check failed. A protected/private note can return 404 to avoid revealing unavailable content. Invalid input returns a safe validation error.

Does email password recovery work offline?

The reset-token and password-change logic exists. Sending a reset email requires configured SMTP; the offline launcher disables SMTP and cloud services. Do not claim automatic offline email delivery.

Files and endpoints to show

src/app/api/[...path]/route.ts
HTTP endpoints: /api/login, /api/upload, /api/notes, /api/collections and /api/admin.

src/lib/security.ts + src/lib/validation.ts
Sessions, role/access checks, origin validation, rate limiting and Zod schemas.

src/lib/storage.ts + src/lib/db.ts + src/lib/notes.ts
Private bytes, database connection helpers and note search/detail queries.

Key distinction: This app uses custom session authentication. It does not use Supabase Auth, Google OAuth or JWT login.